TU Dortmund

Fachprojekt Detektion von Sicherheitslücken

M.Sc. Johannes Düsing, M.Sc. Anemone Kampkötter

JProf. Dr.-Ing. Ben Hermann

Aufgabe 3: Aufruf von Third Party Libraries

Ausgabe: 06.11.2024

Besprechung: 20.11.2024

Third Party Libraries(TPLs) sind ein bedeutendes Konzept in der Software-Entwicklung, das die Wiederverwendung von Funktionalität in Form von externen Bibliotheken möglich macht. Solche Bibliotheken werden in sog. Komponenten-Ökosystemen wie *Maven* zur Verfügung gestellt und können darüber in ein Softwaresystem - als sogenannte *Abhängigkeit* - eingebunden werden.

Diese Abhängigkeiten werden häufig in sogenannten *Buildfiles* spezifiziert, um die benötigten TPLs automatisch herunterladen zu können. Listings 1 und 2 illustrieren wie solche Buildfiles für Maven und SBT aussehen können.

```
<dependencies>
<dependency>
<groupId>ch.qos.logback</groupId>
<artifactId>logback-classic</artifactId>
<version>1.4.4</version>
</dependency>
<dependency>
<groupId>org.apache.maven.indexer</groupId>
<artifactId>indexer-reader</artifactId>
<version>6.0.0</version>
</dependency>
</dependencies>
```

Listing 1: Definition von Abhängigkeiten eines Java Programms in Maven (`pom.xml`)

```
libraryDependencies += "de.opal-project" % "framework_2.13" % "5.0.0"
```

Listing 2: Definition von Abhängigkeiten eines Scala Programms in SBT (`build.sbt`)

Ein Projekt nutzt häufig nur einen kleinen Teil des APIs einer TPL, da die Bibliotheken häufig eine große Menge verschiedener Funktionalitäten zur Verfügung stellen. Dies führt dazu, dass Entwickelnde sehr viel Code herunterladen, der für die Ausführung des Programms nicht benötigt wird und damit redundant ist. Zudem kann der TPL Code unbekannte Schwachstellen enthalten, darum gilt: Um ein Softwaresystem vollständig zu analysieren, ist es nötig, auch die Verwendung der über Third Parties eingebundenen Funktionalität zu analysieren.

3.1 Analyseaufgabe

Implementieren Sie eine statische Programmanalyse, welche die Aufrufe eines gegebenen zu analysierenden Softwaresystems an seine TPLs untersucht. Die Analyse soll für *jede direkt genutzte TPL* ausgeben, wie hoch der Anteil der aufgerufenen Methoden gegenüber den angebotenen Methoden ist. Berechnen Sie außerdem für *jede direkt genutzte TPL*, wie viel von deren Schnittstelle (API) im Kontext des analysierten Systems benutzt wird.

Dafür muss ihre Analyse eine Eingabe erlauben, die genau spezifiziert welche Third Party Libraries Sie (neben dem eigentlichen Softwaresystem) einbinden und analysieren möchten, und wo auf Ihrer Maschine diese zu finden sind.

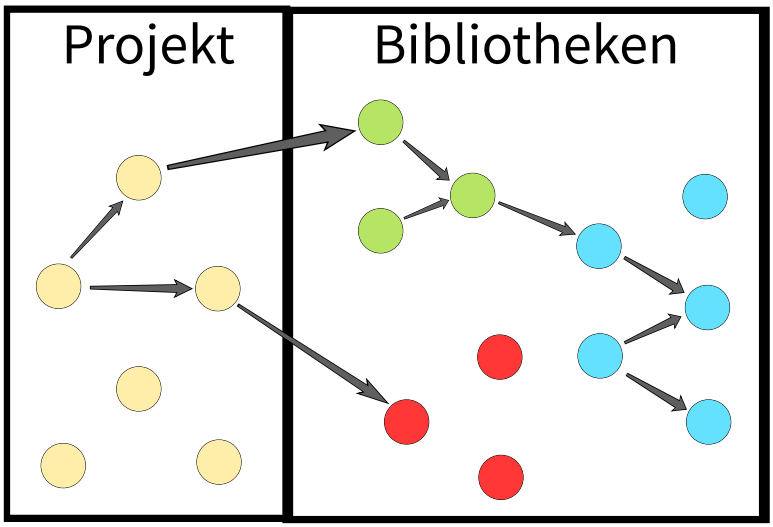

Abbildung 1: Beispiel für einen Whole-Program-Callgraphen

Für eine solche Analyse bieten sich Callgraphen an, ein Beispiel für einen Callgraphen mit TPLs (*Whole-Program-Callgraph*) findet sich in Abbildung 1. Ihre Implementierung soll so funktionieren, dass verschiedene Algorithmen zur Konstruktion von Callgraphen genutzt und von außen konfiguriert werden können. Es soll möglich sein zwischen den folgenden Algorithmen zu wählen:

- CHA
- RTA
- XTA
- 1-1-CFA (`CFA_1_1_CallGraphKey`)

Diese Algorithmen implementieren verschieden Trade-Offs zwischen Laufzeit und Präzision der Ergebnisse. Diesen Sachverhalt sollen Sie in einer Evaluation genauer untersuchen. Stellen Sie dafür einen Benchmark zusammen, der *mindestens drei* Projekte und jeweils deren TPLs beinhaltet. Führen Sie Ihre Analyse für alle Projekte in Ihrem Benchmark *mit jedem möglichen Callgraph Algorithmus* aus. Werten Sie anschließend die folgenden Aspekte aus:

- Wie unterscheiden sich die Ergebnisse für verschiedene Callgraph-Algorithmen voneinander?
- Wie unterscheidet sich die Performance (Laufzeit) der Analyse für verschiedene Callgraph-Algorithmen voneinander?
- Welche Arten von aufgerufenen Methoden werden von den präziseren Callgraph Algorithmen nicht identifiziert, von den unpräziseren aber schon?
- Welcher Algorithmus bietet (für Ihren Benchmark) den besten Trade-Off?

Hinweise:

- Für ein Softwareprojekt mit dem Maven Buildsystem kann man sich durch den Befehl `mvn dependency:tree` alle TPLs ausgeben lassen.
- Experimentieren Sie in Ihrer Analyse mit dem Einbinden verschiedener Third Party Libraries. Die Analyse einzelner Bibliotheken kann die Ressourcen Ihres Rechners unter Umständen überbeanspruchen. Wählen Sie in einem solchen Fall zunächst eine andere Library aus.

- Ausreichend für das erfolgreiche abschließen dieser Aufgabe ist die Betrachtung von direkt abhängige TPLs - das schließt *transitiv genutzte TPLs* (siehe blaue Knoten in Abb. 1) aus. **Wenn Sie möchten:** Erweitern Sie die Analyse so, dass auch berechnet wird welche Methoden aus den transitiv genutzten TPLs aufgerufen werden. Beachten Sie dazu die Code-Beispiele am Ende dieses Zettels.
- Achten Sie beim Messen der Laufzeiten auf eine saubere experimentelle Methodik. Bilden Sie zum Beispiel Mittelwerte über mehrere Läufe, und stellen Sie sicher dass so wenig wie möglich Einfluss von externen Faktoren existiert (Betriebssystem, Hardware, andere Programme) - dafür bietet sich ein Blick auf die Standardabweichung an.
- Das zu analysierende Softwaresystem kann entweder ein Programm (es gibt `main`-Methoden) oder selbst eine Bibliothek sein. Je nachdem müssen Sie den OPAL-Konfigurationswert `org.opalj.br.analyses.cg.InitialEntryPointsKey.analysis` auf den Wert `org.opalj.br.analyses.cg.LibraryEntryPointsFinder` setzen (oder eben nicht).

Technische Rahmenbedingungen:

- Ihre Analyse sollte für Nutzende konfigurierbar sein, sodass der Call-Graph-Algorithmus für das jeweilige Analyseziel gewählt werden kann.
- Stellen Sie sicher, dass Ihre Implementierung die geforderte Funktionalität korrekt umsetzt.
- Stellen Sie uns Ihre Lösung wieder in Ihrem Git-Repository zur Verfügung. Erstellen Sie, wenn möglich und noch nicht geschehen, für die vorherige Aufgabe einen Git-Tag, sodass erkennbar ist, ab wann der Entwicklungsstand dieser neuen Aufgabe beginnt.

Code-Beispiele:

Das folgende Beispiel zeigt, wie die Konfiguration von OPAL beim Erstellen eines Projekts angepasst werden kann, um die Callgraphen für Bibliotheken richtig zu berechnen:

```
1  def loadProject(projectFiles: Array[File],
2    libraryFiles: Array[File], projectIsLibrary: Boolean): Project[URL] = {
3
4      var config = BaseConfig
5
6      if(projectIsLibrary){
7        config = BaseConfig.withValue(
8          "org.opalj.br.analyses.cg.InitialEntryPointsKey.analysis",
9          ConfigValueFactory.fromAnyRef("org.opalj.br.analyses.cg.
               LibraryEntryPointsFinder")
10         ).withValue(
11         "org.opalj.br.analyses.cg.InitialInstantiatedTypesKey.analysis",
12         ConfigValueFactory.fromAnyRef("org.opalj.br.analyses.cg.
               LibraryInstantiatedTypesFinder")
13         )
14     }
15
16     Project(projectFiles, libraryFiles, GlobalLogContext, config)
17 }
```

Das folgende Beispiel zeigt, wie auch die **Inhalte** (also der tatsächliche Code) von den Library-Dateien geladen wird, nicht nur (wie beim Beispiel oben) die *Schnittstelle* der Libraries.

```
1  def loadProjectWithLibraryImplementations(projectFiles: Array[File],
2    libraryFiles: Array[File], config: Config): Project[URL] = {
3
4      val libraryClassFiles = Java17Framework.AllClassFiles(libraryFiles)
5      val projectClassFiles = Java17Framework.AllClassFiles(projectFiles)
6
7      Project(projectClassFiles, libraryClassFiles,
           libraryClassFilesAreInterfacesOnly = false,
8        virtualClassFiles = Iterable.empty)(config)
9  }
```

Das folgende Beispiel zeigt, wie ein CHA Callgraph berechnet werden kann.

```
1 val project = loadProject(/*[...]*/)
2 val callgraph = project.get(CHACallgraphKey)
```